```python
# Step 1: Install the necessary library from Hugging Face
!pip install transformers

# Step 2: Import all required tools
import pandas as pd
from sklearn.model_selection import train_test_split
from transformers import DistilBertTokenizerFast, DistilBertForSequenceClassification, Trainer, TrainingArguments
import torch
import numpy as np
from sklearn.metrics import accuracy_score

# --- Step 3: Prepare the Dataset ---
# We'll create a sample dataset of movie reviews for this demonstration.
print("--- Preparing a sample dataset ---")
reviews = [
    "I loved this movie, the acting was superb!",
    "A complete waste of time, the plot was terrible.",
    "Absolutely fantastic, I would watch it again and again.",
    "I'm not sure why this movie gets so much hate, it was pretty good.",
    "The worst film I have ever seen in my life.",
    "An instant classic, beautifully shot and wonderfully acted.",
    "It was okay, but not something I'd recommend.", # Borderline case
    "So boring I fell asleep halfway through.",
    "A masterpiece of modern cinema.",
    "The script was a mess and the special effects were laughable."
]
# We label the reviews: 1 for Positive, 0 for Negative
labels = [1, 0, 1, 1, 0, 1, 0, 0, 1, 0]
```

```python
# Split our small dataset into a training set and a validation set
train_texts, val_texts, train_labels, val_labels = train_test_split(reviews, labels, test_size=0.2, random_state=42)

# --- Step 4: Load the BERT Tokenizer ---
# The tokenizer converts our text into a format the BERT model can understand.
# 'distilbert-base-uncased' is a smaller, faster version of BERT, ideal for this task.
print("\n--- Loading Tokenizer ---")
tokenizer = DistilBertTokenizerFast.from_pretrained('distilbert-base-uncased')

# Tokenize the training and validation text
train_encodings = tokenizer(train_texts, truncation=True, padding=True)
val_encodings = tokenizer(val_texts, truncation=True, padding=True)

# --- Step 5: Create a PyTorch Dataset Object ---
# This class formats our data in a way that the Hugging Face Trainer can use.
class IMDbDataset(torch.utils.data.Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
        return len(self.labels)


train_dataset = IMDbDataset(train_encodings, train_labels)
```

```
train_dataset = IMDbDataset(train_encodings, train_labels)
val_dataset = IMDbDataset(val_encodings, val_labels)

# --- Step 6: Load the Pre-trained BERT Model ---
# Load the DistilBERT model with a classification head ready for fine-tuning.
print("\n--- Loading Pre-trained Model ---")
model = DistilBertForSequenceClassification.from_pretrained('distilbert-base-uncased', num_labels=2) # 2 labels: Positive/Negative

# --- Step 7: Define Training Arguments and the Trainer ---
# These settings control the fine-tuning process.
training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    logging_dir='./logs',
)

# A function to calculate accuracy during evaluation
def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    acc = accuracy_score(labels, preds)
    return {'accuracy': acc}

# Create the Trainer instance, which handles the entire training loop
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
```

```python
# Create the Trainer instance, which handles the entire training loop
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics
)

# --- Step 8: Start Fine-Tuning the Model ---
print("\n--- Starting Model Fine-Tuning ---")
trainer.train()
print("--- Fine-Tuning Finished ---")


# --- Step 9: Evaluate the Final Model ---
print("\n--- Evaluating Final Model Performance ---")
evaluation_results = trainer.evaluate()
print(f"Final model accuracy on validation set: {evaluation_results['eval_accuracy']:.2f}")

# --- Step 10: Make a Prediction on a New Review ---
print("\n--- Making a Prediction on New Text ---")
new_review = "The movie was not bad, actually it was quite good!"
inputs = tokenizer(new_review, return_tensors="pt", truncation=True, padding=True)

# Put the model in evaluation mode
model.eval()

# Move inputs to the same device as the model (GPU if available)
inputs = {k: v.to(model.device) for k, v in inputs.items()}
```

```python
with torch.no_grad():
    logits = model(**inputs).logits

predicted_class_id = logits.argmax().item()
prediction = "Positive" if predicted_class_id == 1 else "Negative"

print(f"Review: '{new_review}'")
print(f"Predicted Sentiment: {prediction}")
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (4.57.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.20.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.35.3)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2024.11.6)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers) (2.32.4)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.1)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.6.2)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0->transformers) (2025.3.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0->transformers) (4.15.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0->transformers) (1.1.10)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2025.10.5)
--- Preparing a sample dataset ---

--- Loading Tokenizer ---

--- Loading Pre-trained Model ---
```

```
--- Loading Pre-trained Model ---
model.safetensors: 100% 268M/268M [00:03<00:00, 105MB/s]
Some weights of DistilBertForSequenceClassification were not initialized from the model checkpoint at distilbert-base-uncased and are newly initialized: ['classifier.bias', 'class
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

--- Starting Model Fine-Tuning ---
/usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxWarning: invalid escape sequence '\/'
  | |_| | '_ \/ _` / _` |  _/ -_)
wandb: Logging into wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: You can find your API key in your browser here: https://wandb.ai/authorize?ref=models
wandb: Paste an API key from your profile and hit enter: ·····
wandb: WARNING If you're specifying your api key in code, ensure this code is not shared publicly.
wandb: WARNING Consider setting the WANDB_API_KEY environment variable, or running `wandb login` from the command line.
wandb: No netrc file found, creating one.
wandb: Appending key for api.wandb.ai to your netrc file: /root/.netrc
wandb: Currently logged in as: haseena0705 (haseena0705-nitte-meenakshi-institute-of-technology) to https://api.wandb.ai. Use `wandb login --relogin` to force relogin
Tracking run with wandb version 0.22.2
Run data is saved locally in /content/wandb/run-20251019_125747-r7jeuipw
Syncing run spring-durian-1 to Weights & Biases (docs)
View project at https://wandb.ai/haseena0705-nitte-meenakshi-institute-of-technology/huggingface
View run at https://wandb.ai/haseena0705-nitte-meenakshi-institute-of-technology/huggingface/runs/r7jeuipw
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:666: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memor
  warnings.warn(warn_msg)
[6/6 00:15, Epoch 3/3]

 Step  Training Loss

--- Fine-Tuning Finished ---

--- Evaluating Final Model Performance ---
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:666: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memor
  warnings.warn(warn_msg)
```

```
wandb: You can find your API key in your browser here: https://wandb.ai/authorize?ref=models
wandb: Paste an API key from your profile and hit enter: ·····
wandb: WARNING If you're specifying your api key in code, ensure this code is not shared publicly.
wandb: WARNING Consider setting the WANDB_API_KEY environment variable, or running `wandb login` from the command line.
wandb: No netrc file found, creating one.
wandb: Appending key for api.wandb.ai to your netrc file: /root/.netrc
wandb: Currently logged in as: haseena0705 (haseena0705-nitte-meenakshi-institute-of-technology) to https://api.wandb.ai. Use `wandb login --relogin` to force relogin
Tracking run with wandb version 0.22.2
Run data is saved locally in /content/wandb/run-20251019_125747-r7jeuipw
Syncing run spring-durian-1 to Weights & Biases (docs)
View project at https://wandb.ai/haseena0705-nitte-meenakshi-institute-of-technology/huggingface
View run at https://wandb.ai/haseena0705-nitte-meenakshi-institute-of-technology/huggingface/runs/r7jeuipw
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:666: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memor
  warnings.warn(warn_msg)
[6/6 00:15, Epoch 3/3]

 Step  Training Loss

--- Fine-Tuning Finished ---

--- Evaluating Final Model Performance ---
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:666: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memor
  warnings.warn(warn_msg)
[1/1 : < :]
Final model accuracy on validation set: 0.50

--- Making a Prediction on New Text ---
Review: 'The movie was not bad, actually it was quite good!'
Predicted Sentiment: Positive
```